Projukti Sheba

ENGINEERING HIRING ASSESSMENT BRIEF

Position: Mid-Level Laravel / Backend Engineer
Expected Effort: 24–48 Hours
Real-World API & Webhook Automation

01. Challenge Objective

Build a standalone Laravel application titled **"Meta Webhook & Automated Lead Ingestion Engine"**. The goal is to automatically capture customer inquiries from Meta (Facebook Messenger & Lead Ads), verify payload authenticity, prevent duplicates, distribute leads across sales agents, and view them on a clean management dashboard.

02. Core Functional Requirements

A Meta Webhook Handshake & Verification (GET)

- Implement the Meta Webhook Verification endpoint: `GET /api/v1/webhooks/meta`.
- Validate `hub.mode` and `hub.verify_token` against a configured secret in `.env`.
- Return the exact `hub.challenge` string with **HTTP 200 OK** when valid; return **HTTP 403 Forbidden** on mismatch.

B Secure Payload Ingestion & Signature (POST)

- Handle incoming event notifications: `POST /api/v1/webhooks/meta`.
- **Security:** Verify the `X-Hub-Signature-256` header using HMAC-SHA256 with the app secret to guarantee genuine Meta delivery.
- Parse incoming customer data (Sender ID, Name, Phone/Email, Message text, or Lead Ad form fields).

C Anti-Duplicate (Idempotency) Logic

- If the same customer (matching phone, email, or sender ID) sends multiple messages within **24 hours**, do NOT create duplicate leads.
- Instead, append the new message as an **Activity Note** under the existing lead record.
- Ensure data integrity using Eloquent relationships & database transactions.

D Round-Robin Lead Assignment & Dashboard

- **Round-Robin Allocation:** Auto-assign newly created leads evenly among active sales agents (users with a Sales role).
- **Lead Explorer Dashboard:** A clean UI displaying incoming leads, source badges (Messenger / Lead Ad), assigned agent, and activity logs.

03. Architecture & Conventions

- **Environment Configuration:** `META_VERIFY_TOKEN` and `META_APP_SECRET` in `.env.example`.
- **Service Layer:** Separate webhook handling into dedicated Service classes (e.g., `MetaWebhookService`) to keep controllers thin.
- **Response Discipline:** Always acknowledge valid webhooks immediately with `HTTP 200 {"status": "EVENT_RECEIVED"}`.
- **Database Design:** Relational schema linking Leads, LeadActivities, and Assigned Users.

04. Bonus / Standout Expectations

- **Queued Processing:** Dispatch webhook payload to a background Queue Job to respond within <200ms.
- **Mock Webhook Tester:** Provide a sample cURL command or Postman collection with realistic Meta JSON payloads for testing.
- **Automated Feature Test:** 1 or 2 PHPUnit/Pest tests simulating valid and invalid webhook requests.

05. Submission Instructions

- **Repository:** Push code to a public GitHub / GitLab repository with clear, iterative commit history.
- **Documentation:** Include a comprehensive `README.md` with setup instructions, migration steps, and sample cURL test payloads.
- **Delivery:** Share the repository URL with our hiring team upon completion.